

Architecture

BANK-SPEC is one Clojure/JVM application in memory, without a web layer, database or UI. This document, tradeoffs and production considerations form the PDF addressing the Part 2 requirements.

Domains and module boundaries

- **Authorization** owns operational transactions, available funds, holds, decisions and snapshots. Transaction entry calls only Authorization.
- **Ledger** owns accounting balances and journals, deduplicated by financial transaction ID. Balanced postings use customer liabilities and same-currency clearing. Opening balances remain configuration.
- **Yield and Fees** owns assessments, accruals, financial intents and settlement receipts. It reconstructs dated bases from approved events and opening state. Neither it nor Authorization reads Ledger balances.

Each domain separates pure logic, incoming/outgoing ports, atomic local storage in db, and internal schemas in model, mirrored in tests. Ledger has no outgoing application dependency. Shared arithmetic and validation remain pure.

Snapshots and concurrency

Recorded IDs, including declines, win before content validation; duplicates repeat no effect. Financial and hold changes create snapshots, advancing the counter by one from 1; declines record only identity/decision. Check identity and base version atomically with recording the outcome/effects. A changed base requires recalculation with the same ID. No module lock spans another module call; uncommitted attempts publish nothing.

Calculation version validation

Unrecorded interest and fee commands require `source_event_counter` equal to Authorization's current counter. For source 10 against current 11, record no payment; receive event 11 and recalculate with the same unrecorded ID. Never replace only the counter. A financial booking cutoff does not remove this version check.

Yield saves the complete command before submission: ID, amount, dates, source, component links and fee assessment where applicable. Unknown outcomes retain that exact command for confirmation or identical retry. Definite invalid/state rejection permits a fresh proposal while preserving intent history. Confirmed results, confirmed duplicates or committed events atomically record the original assessment/receipt and clear pending state before another settlement selects components. Pending commands block other Yield fees/payments, including zero settlements, but not principal credits/debits. Jobs are serial; each fee is confirmed and delivered before the next proposal.

Delivery, time and reports

The dispatcher sends recorded envelopes to Yield and financial events to Ledger, acknowledging recipients independently. Explicit drains retry failures. Completeness requires the contiguous approved prefix, including holds, no pending financial command and no known pending delivery; this cannot prove future external completeness. Recovery ends at process exit.

Value date controls economic effect; booking date controls input eligibility; account counter/journal position bounds observed state. Receipt is separate. Daily D jobs use bookings through D-1 and effects through the calculated day. Corrections append linked differences; pending interest is outside balances and earns nothing. Captured reports stay immutable. Live cross-module reads are non-atomic and trigger no recovery; historical accounting uses explicit date/position bounds.

Example 08 alone permits a principal-input bound with `:interest-only? true`, without fee proposals. Default jobs use all eligible inputs.

See API and CONTRACTS for interfaces; NUMBERS and AMBIGUITIES for financial rules and decisions.

C3 component view

This logical C3 (C4 Level 3) view shows components inside one application. Numbers label interactions, not event counters or execution order. Calculation has an independent trigger.

Transaction to Authorization: 1. Submit transactions

Authorization to Yield and Fees: 2. Send approved transactions after recording

Authorization to Ledger: 3. Send financial events after recording

Yield and Fees to Authorization: 10. Submit interest and fees with source counter

Authorization to Yield and Fees: 11. Approves interest and fee or request recalculation when source counter mismatch

Tradeoffs

These implemented choices favor a small, explainable model. Production limits below describe work still required, not delivered capabilities.

Choice	Benefit	Cost or boundary
One application, three modules, explicit ports	Clear financial ownership without distributed infrastructure.	Shared failure and memory limits; ports provide no security isolation.
Pure rules, local atomic writes, serial jobs	Testable decisions; identity, version and recording stay together.	Commands serialize locally; concurrent Yield jobs are unsupported.
Append-only history and corrections	Reproduce earlier views and explain later changes.	Unbounded history and repeated scans.
Save financial intent before submission	Retry the original command; reconcile its original components.	New Yield fees or payments wait; principal continues. No restart recovery.
Recorded ID wins without payload comparison	Retries repeat no effect.	A reused ID can hide different content; the separate challenge demonstrates this limitation.
Reconstruct dated balances from approved events	Explain late corrections without consuming another module's balance.	Scanning costs; a complete internal prefix cannot prove all external inputs arrived.
Exact decimals and daily currency rounding	Reconcile posted amounts and rounded components exactly.	Currency scales, daily rate and AED fee come from the exercise; HALF_UP and the BHD fee exception are adopted choices.

What was cut and why

The first two rows are explicit exercise exclusions. The remaining rows are BANK-SPEC choices or limits of the supplied scenario. Earlier choices also exclude parallel jobs and payload-conflict detection.

Cut or simplification	Why retained	Production risk deferred
Durable storage and database	Required in-memory scope.	State, pending intents and audit records vanish on restart.
HTTP/web layer and UI	Required core-only scope.	No customer interface, authenticated access or authorization of external callers.
Broker, distribution, timers, backoff and rejected-message queue	Deterministic delivery and explicit retry keep coordination small.	No independent failover or automatic recovery; failures need operator action.
Payment-network integration and external completeness proof	Inputs are assumed legitimate; only known internal delivery is checked.	Missing or disputed external money needs authentication, reconciliation and exception handling.
Automatic hold expiry	No contractual lifetime supplied.	Uncollected holds can remain indefinitely; production needs audited expiry releases.
Real business calendar	Integer days make the fixture reproducible.	Holiday, time-zone and cutoff mistakes can change financial eligibility.
General installment schedule	One supplied three-part credit is one atomic journal.	No independent future installment posting or collection lifecycle.
Production chart of accounts	Same-currency clearing balances this small ledger.	Fee revenue, interest expense and settlement controls need dedicated accounting mappings.
FX and yield taxes	No conversion or tax policies adopted.	Cross-currency valuation and required withholding/reporting are unimplemented.
Bank-caused error remediation	Corrections cover legitimate late transactions and reversals only.	Customer redress can require treatment different from regular pending-interest payment.
General historical-view engine	One explicit interest-only bound serves the supplied correction example.	Arbitrary partial views and partial-view fees are unsupported.

Production Considerations

These are limits and recommendations, not implemented production capabilities.

Growth at 100 times the volume

With a drain after each transaction, the first avoidable latency cost to address is delivery lookup: every drain scans retained :delivery-order, including acknowledged IDs. Acknowledgement removes the envelope from :deliveries; order and acknowledgement IDs remain. Repeated drains can therefore accumulate quadratic work. Heap capacity may become the first hard limit; no benchmark establishes a universal bottleneck ranking.

Unbounded state includes Authorization's identity results, decision/snapshot history and delivery indexes; Ledger's journal and identity set; and Yield's events, fee/interest components, receipts, intent history and identity indexes. Ledger queries scan history; Yield repeatedly scans events/components. At 100 times the retained records, a linear scan examines approximately 100 times as many records. This is an algorithmic estimate; Clojure collections share structure.

Cheapest change: maintain a separate ordered index of pending deliveries, removing acknowledged entries from that work index while preserving financial/audit history. Next, add per-account journal indexes and dated checkpoints, invalidated by relevant late entries and preserving booking, value and observation boundaries. Indexes reduce scans; durable storage and a governed archive policy are still needed to bound live memory. Deleting old identities or settled components can break duplicate protection or later corrections.

Authorization endings and intermediate states

These are the current model's complete lifecycle paths. An already approved hold can end through full explicit release, final capture, or a capture that exhausts its remainder. Decline is an initial refusal. A reversal corrects money and does not itself terminate or restore a hold.

Real-world situation	Required model behavior
A purchase request exceeds available funds	Record a permanent decline without creating a hold, financial posting or snapshot. Later funds do not reactivate it.
A merchant cancels the remaining order	Full explicit release ends the hold and restores availability without crediting money.
A hotel's final bill is below its reservation	Debit the final bill, release the unused remainder and end the hold without a release credit.
A restaurant's confirmed final bill, including a tip, consumes or exceeds the reservation	Debit the whole confirmed amount and exhaust the hold, including with a non-final flag. Do not suppress confirmed money because availability is insufficient.
A merchant sends a late capture after cancellation, decline or an earlier exhausting capture	Record the debit and inactive-authorization; create no new hold. A never-recorded reference produces missing-authorization.
A captured charge is fully reversed	Append the principal reversal once. Preserve any residual hold; do not recreate a consumed or released hold.

For an order split across shipments, a partial cancellation releases only that part; a non-final capture below the remaining hold consumes only the captured amount. Both are intermediate states. Invalid releases leave the existing state unchanged, and duplicate transaction IDs repeat no effect.

Expiry is not implemented: if a hotel never captures or releases a reservation, its residual hold remains active. Before production, define the contractual deadline and append an audited expiry release, restoring availability without a financial credit. Do not infer a deadline from the replay window. A genuine late capture still requires recording and reconciliation.

Value dates for a UAE licensed bank

A late booking can change an earlier economic balance and its fee/interest entitlement while preserving a previously captured report. Operations must reconcile postings, recalculated entitlements and affected statements; retain both dates and the observation boundary; and control changes affecting closed periods. A backdated economic effect is not evidence that funds were operationally available earlier.

For applicable consumer deposit products, CBUAE Standards 2.1.2.4-5 cover interest/payment and fee disclosures; 2.1.2.9 covers periodic statements with movements, interest, fees and closing balance. These duties make value-date corrections a customer communication and statement issue, not just arithmetic. They do not establish this exercise's cutoff, rounding or fee policy. [CBUAE, pages 9-10](#).

Bank-caused errors are outside this implementation's correction scope. CBUAE 5.1.1.38-40 requires correction for affected consumers, immediate refunds of deductions/costs caused by an error, and notification within ten complete business days of identifying it. The exercise's regular pending-interest schedule must not be assumed sufficient for that redress. [CBUAE, page 45](#).

One control before live use: an independently approved backdated-adjustment gate. Record the authenticated initiator and approver, original reference, cause classification, both dates and a preview of affected balances, fees and interest; reconcile postings and statements before release. Route bank-caused errors to the applicable redress process and flag closed-period changes. This is a proposed implementation of control, not a claim that the cited clauses prescribe that exact workflow. Applicability depends on the institution, customer and product; durable recovery and a contractual calendar remain prerequisites.